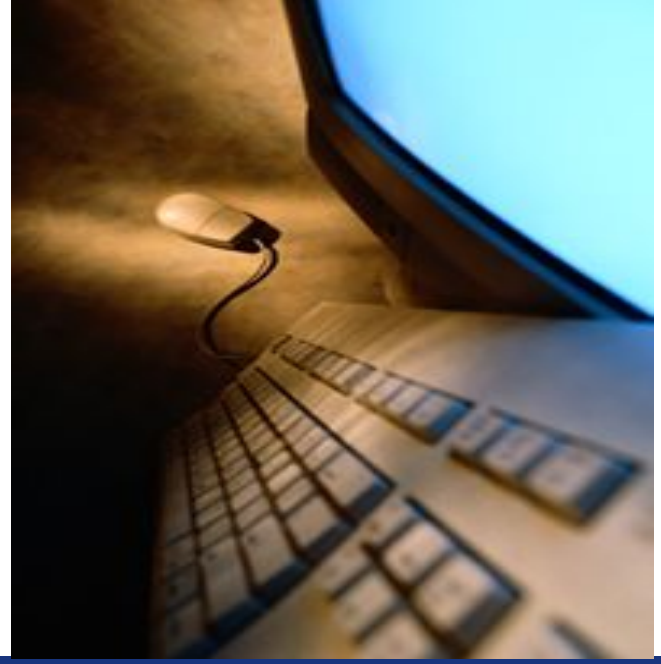
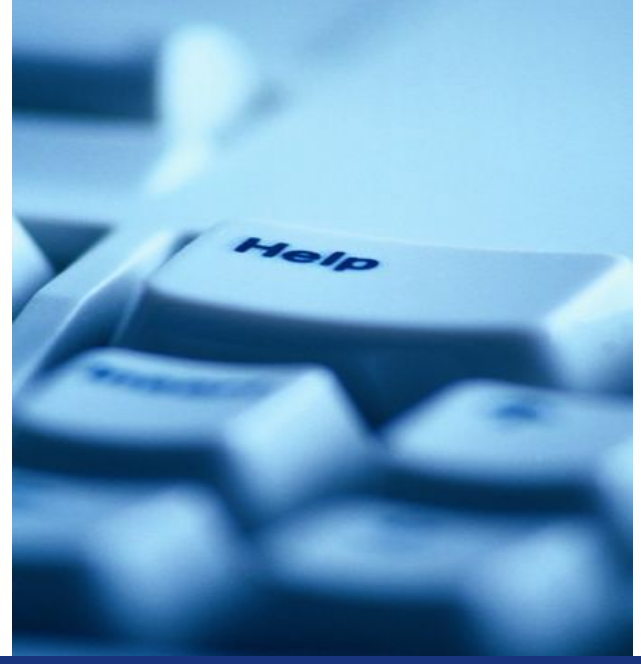
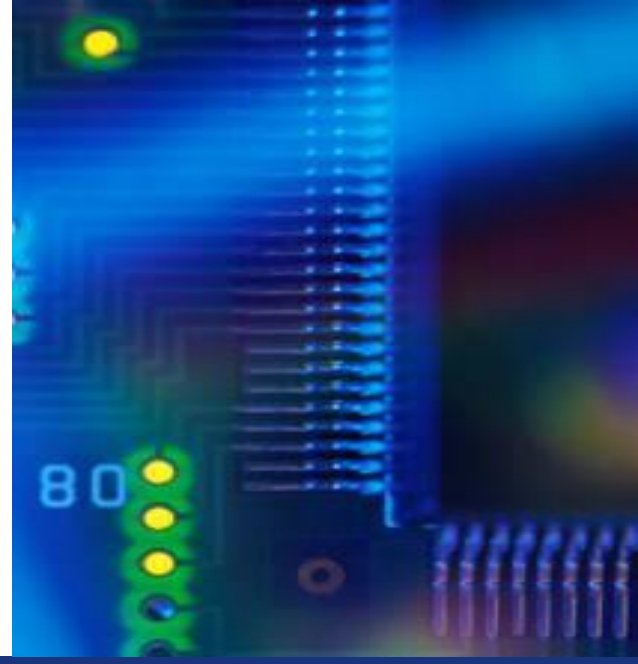
UNIVERSITAS
BUDI LUHUR







FAKULTAS TEKNOLOGI INFORMASI

Pemrograman Java Web dan Enterprise

[PG191/ 3 SKS]

Semester Ganjil 2026/2027

Dosen : Riskiana Wulan S.Kom, M.Kom







Pertemuan 5
Building Basic REST API with Spring Boot






Learning Objectives & Prerequisite

- **Setelah mengikuti perkuliahan ini, mahasiswa mampu:**
  - Memahami arsitektur REST (Representational State Transfer) dan peran format JSON sebagai standar pertukaran data pada Web API modern.
  - Membangun kelas REST Controller menggunakan anotasi `@RestController` dan memetakan HTTP Methods (GET, POST, PUT, DELETE).
  - Menerapkan anotasi ekstraksi masukan HTTP request: `@PathVariable`, `@RequestParam`, dan `@RequestBody`.
  - Membangun layanan CRUD (Create, Read, Update, Delete) katalog produk berbasis memori (In-Memory).
  - Menguji seluruh endpoint REST API menggunakan aplikasi Postman.

- **Prerequisite:**
  - Mahasiswa telah menyelesaikan materi Week 4: memahami konsep Inversion of Control (IoC) dan Dependency Injection (DI), serta pembuatan Spring Beans (`@Repository` dan `@Service`) dalam project Spring Boot.






Big Picture

- **Dari Server-Side Rendering ke REST API**
  - Pada minggu-minggu sebelumnya, aplikasi kita menghasilkan tampilan HTML mentah dari sisi server (Server-Side Rendering).
  - Dalam arsitektur enterprise modern (Backend-Frontend Decoupled), aplikasi Java Backend bertugas sebagai penyedia data mentah berformat JSON melalui REST API, yang dapat dikonsumsi aplikasi web React/Vue, aplikasi mobile Android/iOS, maupun sistem pihak ketiga.
  - Pada Minggu 5, kita melengkapi arsitektur `product-catalog` dengan Controller Layer (`@RestController`) berisi endpoint CRUD lengkap, lalu mengujinya secara interaktif menggunakan Postman.






REST Architecture & JSON Payload

Representational State Transfer (REST) adalah gaya arsitektur perangkat lunak (*architectural style*) yang mendefinisikan sekumpulan aturan dan batasan (*constraints*) untuk membangun layanan web (*web services*).

Dikenalkan pertama kali oleh Roy Fielding pada tahun 2000, REST dirancang agar komunikasi antar sistem di internet berjalan secara efisien, terstruktur, dan memiliki skala yang tinggi (*scalable*).

REST API (*RESTful Application Programming Interface*) adalah implementasi antarmuka pemrograman aplikasi yang menerapkan prinsip-prinsip arsitektur REST. REST API memungkinkan dua sistem berkomunikasi melalui protokol HTTP dengan menggunakan format data seperti JSON atau XML.

- **JSON adalah standar REST API karena ringan (lightweight), berstruktur key-value, dan mudah dibaca berbagai bahasa pemrograman.**
  - Spring Boot melakukan Serialization (Objek Java ke JSON) dan Deserialization (JSON ke Objek Java) secara otomatis melalui library Jackson.

```
{
  "id": "PRD-01",
  "name": "Keyboard Mechanical",
  "price": 450000.0
}
```

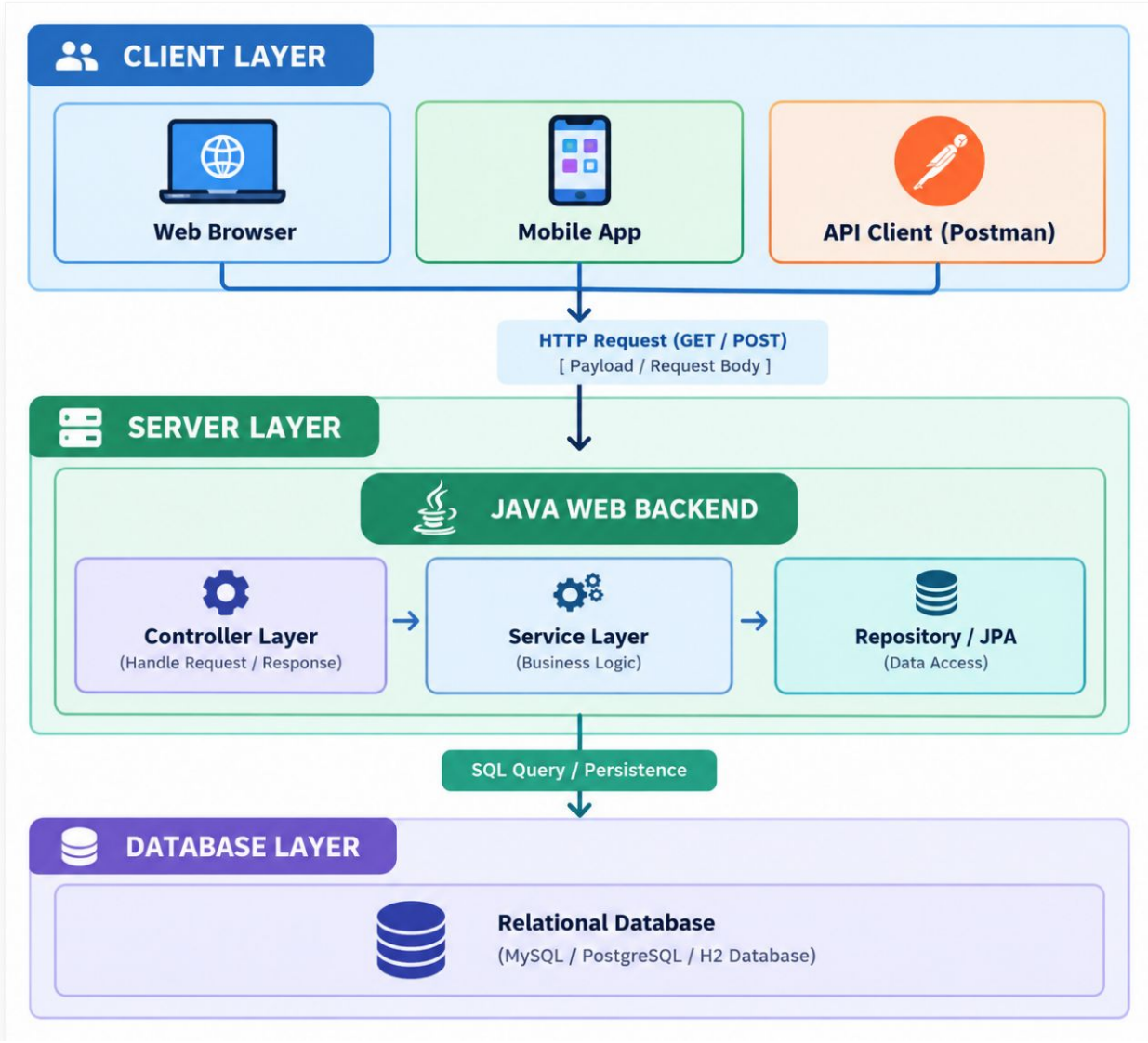





Konsep Komponent dalam REST API

- **Resource:** Objek atau data yang dikelola oleh sistem (misalnya: *User*, *Product*, *Order*).
- **Endpoint (URI):** Alamat URL yang digunakan untuk mengakses *resource*.
- **HTTP Methods:** Kata kerja standar HTTP untuk menentukan aksi pada *resource*:
  - GET: Mengambil data *resource*.
  - POST: Membuat *resource* baru.
  - PUT: Mengganti seluruh data *resource* yang ada.
  - PATCH: Perbarui sebagian data *resource*.
  - DELETE: Menghapus *resource*.

Diagram Arsitektur Client-Server (Backend-Frontend)



- **Flow**:
  1. Client mengirimkan HTTP Request (misal: `GET /api/products`).
  2. Java Web Server menerima request, memproses logika bisnis di Service Layer, dan mengambil data dari Database melalui Repository (JPA).
  3. Server mengemas data dalam format JSON dan mengirimkannya kembali sebagai HTTP Response kepada Client.






JSON Data Flow: From Client to Server

Data dikirim dari client ke server dalam format JSON, kemudian diproses oleh backend dan dikonversi menjadi Java Object.

1 Client (Browser / Mobile App / API Client)

- Web Browser
- Mobile App
- API Client (Postman)

Mengirim HTTP Request (dengan payload JSON)

HTTP Request
(POST / GET / PUT / DELETE)
+ JSON Body

2 Network

Internet / Network
(HTTP/HTTPS)

3 Server (Java Web Backend)

- **Controller** (Terima request & parse JSON)
- **Service** (Business Logic)
- **Repository / JPA** (Akses Database)

Java

4 Database

Relational Database
(MySQL / PostgreSQL / H2 Database)

HTTP Response
(JSON)

{ } Format JSON & Mapping ke Java Object

A Contoh JSON Response (dari Server)

```
{
   "id": 1,
   "name": "Budi Santoso",
   "email": "budi@mail.com",
   "course": {
      "id": 10,
      "name": "Java Web Programming"
   }
}
```

JSON menggunakan pasangan **key-value**, dengan struktur seperti **object** dan **array**.

B Di Server (Java)

1. **Controller** menerima request dan membaca JSON body.
2. JSON dikonversi menjadi Java Object (DTO/Entity).
3. **Service** memproses data (business logic).
4. **Repository/JPA** menyimpan atau mengambil data dari database.
5. Hasil diproses menjadi JSON dan dikirim ke client.

C Mapping ke Java Object

Java Class (contoh)

```
public class Student {
   private Integer id;
   private String name;
   private String email;
   private Course course;
}
public class Course {
   private Integer id;
   private String name;
}
```

Objek Java (hasil mapping)

```
Student {
   id: 1,
   name: "Budi Santoso",
   email: "budi@mail.com",
   course: Course {
      id: 10,
      name: "Java Web Programming"
   }
}
```

D Pola / Pattern yang Digunakan

JSON (Format Data) → Jackson (Converter JSON ↔ Java Object) → Java Object (DTO / Entity)

Tips

- Pastikan nama field JSON sama dengan nama atribut Java (atau gunakan @JsonProperty).
- Gunakan DTO untuk memisahkan struktur data dari entity.
- Validasi input JSON agar data lebih aman.






Spring REST Annotations

- **`@RestController`: kombinasi `@Controller` dan `@ResponseBody`. Nilai Response setiap method langsung dituliskan sebagai response body berformat JSON, bukan nama halaman HTML.**
- **`@RequestMapping("/api/products")`: menentukan base URL path utama untuk seluruh endpoint di dalam kelas Controller.**
- **Anotasi pemetaan HTTP Method:**
  - `@GetMapping`: memetakan HTTP GET (membaca/mengambil data).
  - `@PostMapping`: memetakan HTTP POST (membuat/menambah data baru).
  - `@PutMapping`: memetakan HTTP PUT (memperbarui data yang ada).
  - `@DeleteMapping`: memetakan HTTP DELETE (menghapus data).






Extracting Request Data

- **@PathVariable: mengambil nilai yang disisipkan langsung pada URI Path.**
  - Contoh URL: GET /api/products/PRD-01
  - Method Java: public Product getById(@PathVariable("id") String id)
- **@RequestParam: mengambil nilai dari query parameter URL di belakang tanda tanya (?).**
  - Contoh URL: GET /api/products/search?name=keyboard
  - Method Java: public List<Product> search(@RequestParam("name") String name)
- **@RequestBody: membaca JSON payload pada Body pesan HTTP (umum pada POST dan PUT).**
  - Payload Body: { "id": "PRD-03", "name": "Mouse Pad", "price": 50000.0 }
  - Method Java: public Product create(@RequestBody Product product)







API Testing

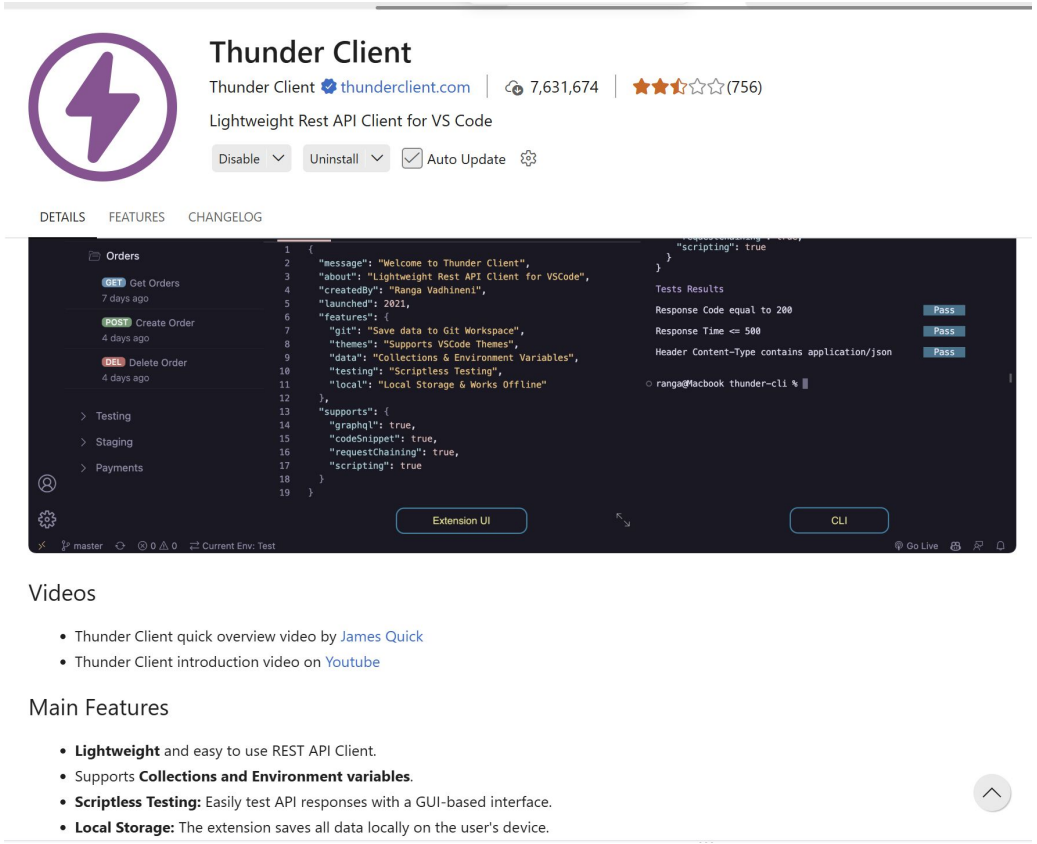
Postman adalah API Testing Tool, yang digunakan oleh developer untuk:

- Mengirimkan request HTTP (GET, POST, PUT, DELETE).
- Mengatur request header (Content-Type: application/json).
- Mengirimkan JSON payload body.
- Menginspeksi HTTP Status Code dan JSON response dari server.

Download Postman:
https://www.postman.com/downloads/



Thunder Client (VS Code Extension for REST API Client



Videos

- Thunder Client quick overview video by James Quick
- Thunder Client introduction video on Youtube

Main Features

- **Lightweight** and easy to use REST API Client.
- Supports **Collections and Environment variables**.
- **Scriptless Testing:** Easily test API responses with a GUI-based interface.
- **Local Storage:** The extension saves all data locally on the user's device.






Hands-on Lab

- **Tujuan Lab:**

1. Membangun ProductController berbasis REST API pada project Spring Boot product-catalog.
2. Melakukan pengujian CRUD lengkap menggunakan Postman.

- **Starting Point: gunakan project product-catalog dari Week 4 (sudah memiliki Product.java, ProductNotFoundException.java, ProductRepository.java, dan ProductService.java).**
- **Catatan: pastikan Product.java memiliki setter setName() dan setPrice() (dibutuhkan untuk fitur update).**






Step 1: Update & Delete Methods

// ProductRepository.java (tambahkan)

```
public Product updateProduct(String id,
        Product updatedProduct)
        throws ProductNotFoundException {
    Product existingProduct = findById(id);
    existingProduct.setName(
        updatedProduct.getName());
    existingProduct.setPrice(
        updatedProduct.getPrice());
    return existingProduct;
}

public boolean deleteById(String id)
        throws ProductNotFoundException {
    Product existingProduct = findById(id);
    return productList.remove(
        existingProduct);
}
```

// ProductService.java (tambahkan)

```
public Product updateProduct(String id,
        Product product)
        throws ProductNotFoundException {
    return productRepository
        .updateProduct(id, product);
}

public boolean deleteProduct(String id)
        throws ProductNotFoundException {
    return productRepository
        .deleteById(id);
}
```






Step 2: Create ProductController (1/3)

```
package com.budiluhur.catalog.controller;

import com.budiluhur.catalog.exception
    .ProductNotFoundException;
import com.budiluhur.catalog.model.Product;
import com.budiluhur.catalog.service
    .ProductService;
import org.springframework.http.HttpStatus;
import org.springframework.http
    .ResponseEntity;
import org.springframework.web.bind
    .annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/products")
public class ProductController {
```

```
private final ProductService
    productService;

// Constructor Injection
public ProductController(
        ProductService productService) {
    this.productService = productService;
}

// 1. GET ALL: GET /api/products
@GetMapping
public List<Product> getAllProducts() {
    return productService
        .getAllProducts();
}
```






Step 2: ProductController (2/3)

```
// 2. GET BY ID: GET /api/products/{id}
@GetMapping("/{id}")
public ResponseEntity<?> getProductById(
        @PathVariable("id") String id) {
    try {
        Product product = productService
            .getProductById(id);
        return ResponseEntity.ok(product);
    } catch (ProductNotFoundException e) {
        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(e.getMessage());
    }
}
```

```
// 3. CREATE: POST /api/products
@PostMapping
public ResponseEntity<String> createProduct(
        @RequestBody Product product) {
    productService.createProduct(product);
    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body("Produk '" + product.getName()
            + "' berhasil ditambahkan!");
}
```






Step 2: ProductController (3/3)

```
// 4. UPDATE: PUT /api/products/{id}
@PutMapping("/{id}")
public ResponseEntity<?> updateProduct(
        @PathVariable("id") String id,
        @RequestBody Product product) {
    try {
        Product updatedProduct =
            productService.updateProduct(
                id, product);
        return ResponseEntity
            .ok(updatedProduct);
    } catch (ProductNotFoundException e) {
        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(e.getMessage());
    }
}
```

```
// 5. DELETE: DELETE /api/products/{id}
@DeleteMapping("/{id}")
public ResponseEntity<String> deleteProduct(
        @PathVariable("id") String id) {
    try {
        productService.deleteProduct(id);
        return ResponseEntity.ok(
            "Produk dengan ID '" + id
            + "' berhasil dihapus!");
    } catch (ProductNotFoundException e) {
        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(e.getMessage());
    }
}
```






Step 3: Running the Application

- **Langkah menjalankan:**

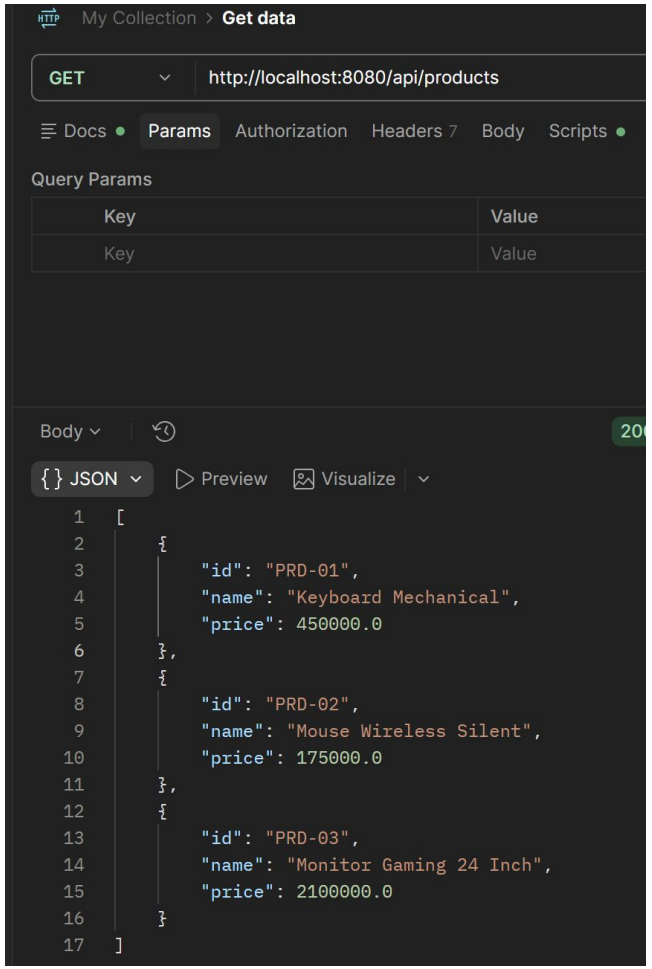
1. Jalankan file `Application.java` dari IDE.
2. Pastikan di log konsol terminal tertera bahwa Tomcat telah berjalan pada port 8080.






Step 4: Testing with Postman (1/4)

- **Test GET All Products:**
  - Method: GET | URL: http://localhost:8080/api/products | Click Send.
  - Expected Result: Status 200 OK dan mengembalikan JSON Array berisi daftar 2 produk awal.

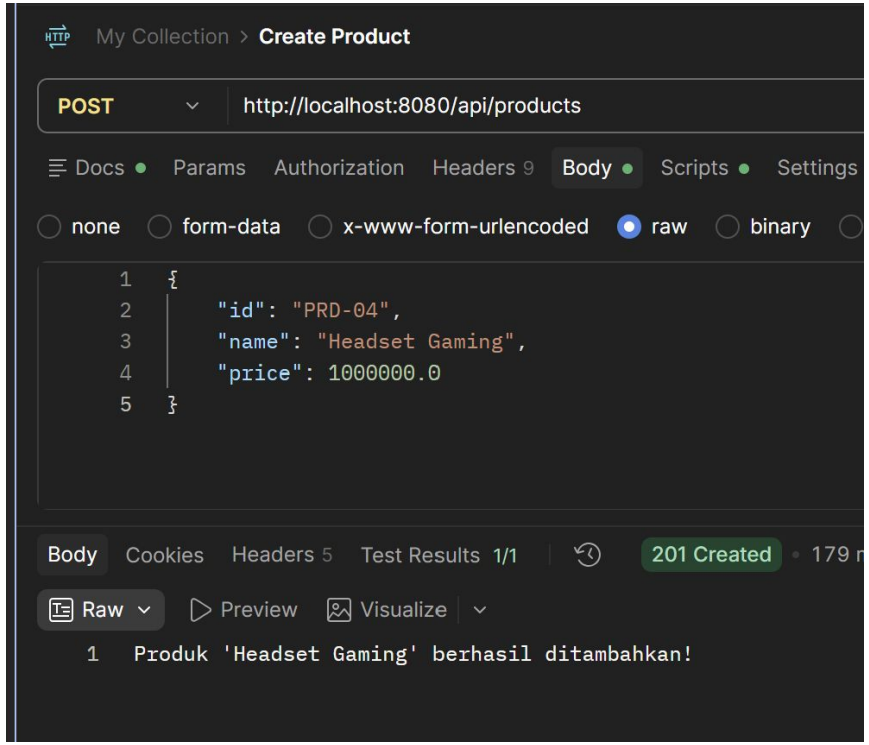


Testing with Postman (2/3)

- **Test POST Create Product:**
  - Method: POST | URL: http://localhost:8080/api/products
  - Tab Headers: set Content-Type = application/json.
  - Tab Body, pilih raw, lalu JSON:

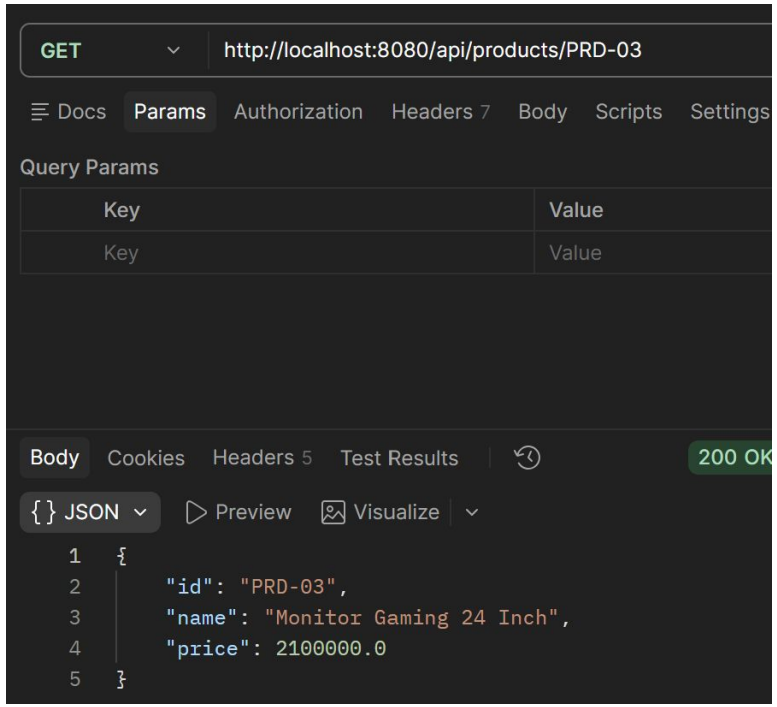
```
{
   "id": "PRD-04",
   "name": "Headset Gaming",
   "price": 1000000.0
}
```

  - Click Send. Expected Result: Status 201 Created dengan respon teks sukses.



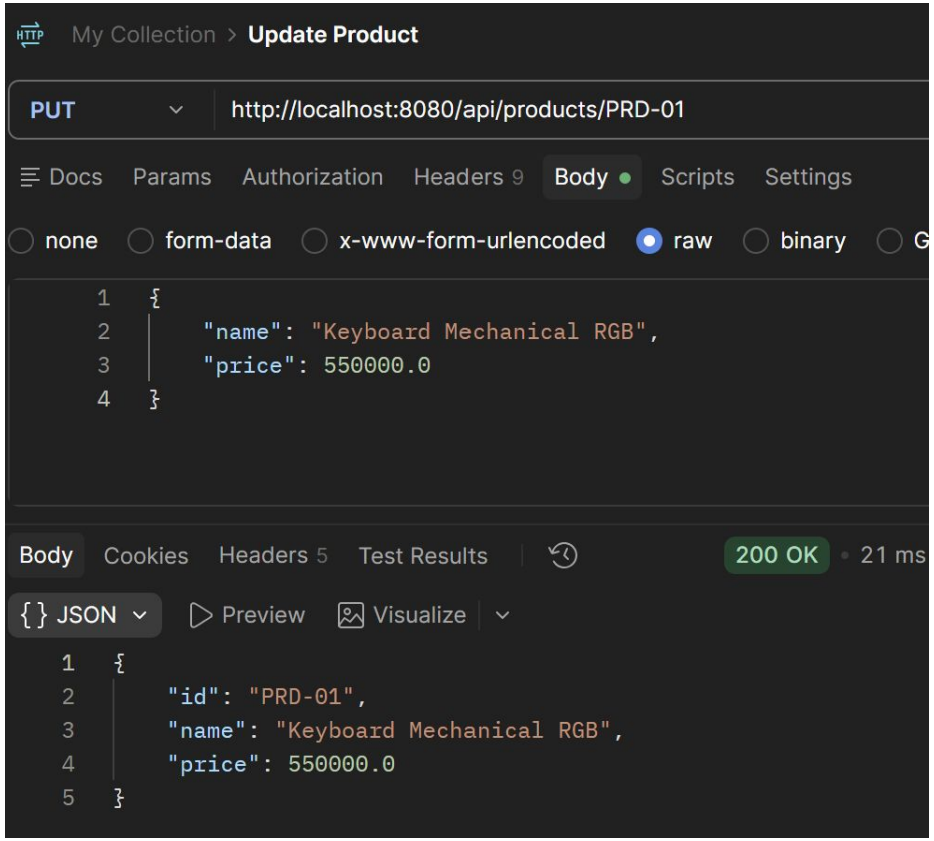
Step 4: Testing with Postman (3/3)

- **Test GET Product By ID (Sukses & Error):**
  - URL Sukses: `GET http://localhost:8080/api/products/PRD-03` - Status 200 OK dan data produk.
  - URL Error: `GET http://localhost:8080/api/products/PRD-99` - Status 404 Not Found dan pesan error.



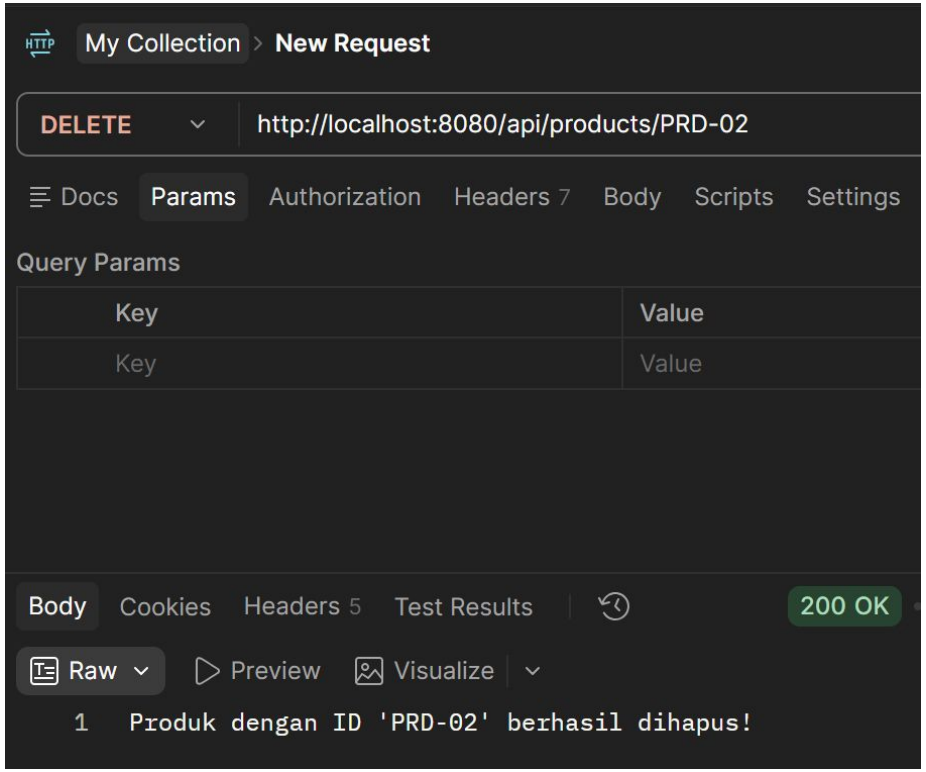
Step 4: Testing with Postman (3/3)

- **Test PUT Update Product:**
  - Method: `PUT` | URL: `http://localhost:8080/api/products/PRD-01`
  - Body JSON: `{ "name": "Keyboard Mechanical RGB", "price": 550000.0 }`
  - Expected Result: Status 200 OK dan JSON data produk yang telah diperbarui.



Step 4: Testing with Postman (3/3)

- **Test DELETE Product:**
  - Method: DELETE | URL: http://localhost:8080/api/products/PRD-02
  - Expected Result: Status 200 OK dengan pesan sukses penghapusan.








Mini Challenge

- **Modifikasi kelas `ProductController.java`:**

1. Buat endpoint baru untuk pencarian produk berdasarkan nama: `GET /api/products/search?keyword=keyboard`.
2. Gunakan anotasi `@RequestParam("keyword") String keyword`.
3. Tambahkan method pencarian pada `ProductService` dan `ProductRepository` yang menyaring produk dengan kriteria `product.getName().toLowerCase().contains(keyword.toLowerCase())`.
4. Test endpoint pencarian di Postman dan pastikan respon mengembalikan daftar produk yang mengandung kata kunci tersebut.






Common Mistakes (1/2)

- **Problem: Error `415 Unsupported Media Type` saat request POST / PUT di Postman.**
  - Penyebab: lupa mengatur header `Content-Type: application/json`, atau memilih format Text/HTML pada tab Body.
  - Cara Memperbaiki: di Postman, pilih tab Body, radio button raw, lalu ubah dropdown dari Text menjadi JSON.
- **Problem: variabel bernilai `null` atau `0.0` pada objek saat diterima di Controller.**
  - Penyebab: lupa menambahkan anotasi `@RequestBody` pada parameter method, sehingga Spring tidak membaca data dari HTTP Body.
  - Cara Memperbaiki: awali parameter dengan `@RequestBody`: `createProduct(@RequestBody Product product)`.






Common Mistakes (2/2)

- **Problem: Error `400 Bad Request` atau `Required request parameter 'xxx' is not present`.**
  - Penyebab: nama pada `@RequestParam("keyword")` tidak cocok dengan query key di URL Postman (`?keyword=...`).
  - Cara Memperbaiki: samakan ejaan parameter antara kode Java dan URL Postman (bersifat case-sensitive).
- **Problem: data buatan di Postman kembali ke data awal setelah aplikasi di-restart.**
  - Penyebab: penyimpanan masih In-Memory (di dalam List), sehingga seluruh data akan hilang saat aplikasi Spring Boot di-restart.






Recap & Preparation for Next Week

- **Recap:**
  - REST API menggunakan protokol HTTP dan format data JSON sebagai standar pertukaran data backend-frontend.
  - `@RestController` menandai kelas pembentuk REST API.
  - HTTP Method mapping: `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`.
  - Ekstraksi masukan: `@PathVariable` (URI Path), `@RequestParam` (?key=value), `@RequestBody` (JSON Body).
  - `ResponseEntity<?>` mengatur data respon beserta HTTP Status Code (200 OK, 201 Created, 404 Not Found).
  - Postman menguji seluruh endpoint REST API secara independen sebelum diintegrasikan ke Frontend.

- **Preparation for Next Week:**
  - REST API produk sudah berfungsi penuh dengan CRUD, namun data masih tersimpan di memori RAM (In-Memory List) dan hilang saat aplikasi dimatikan.
  - Minggu ke-6: Integrasi Database Pertama dengan Spring Data JPA & Hibernate, menghubungkan `product-catalog` ke database relasional (H2/MySQL) agar data tersimpan permanen.



SELESAI